Generative classifiers: the Bayes classifier and naive Bayes

Prof. Dr. Emre Onur Kahya · Department of Physics Engineering, Istanbul Technical University

A classifier can be built from density estimators: model how the data of each class are distributed, then let Bayes' rule turn those densities into class probabilities. This note develops that recipe, shows that Gaussian class densities produce linear or quadratic decision boundaries, explains the strong independence assumption of naive Bayes, and compares the generative route with the discriminative one taken in Note 9. The same reasoning underlies the likelihood-ratio tests of experimental physics and the use of today's generative models as classifiers.

1 From scores to decisions

Most classifiers first compute real-valued *scores* and then decide. In the binary case a single *discriminant function* $g(\mathbf{x})$ is compared with a threshold: predict class 1 if $g(\mathbf{x}) > \tau$. Common choices are $\tau = 0$ for an unbounded score and $\tau = \frac{1}{2}$ when $g(\mathbf{x})$ estimates $P(Y = 1 \mid \mathbf{x})$. With m classes the model outputs a vector of scores $\mathbf{g}(\mathbf{x}) \in \mathbb{R}^m$ and predicts $f(\mathbf{x}) = \arg\max_c g_c(\mathbf{x})$; for $m = 2$ this is the sign of $g_2(\mathbf{x}) - g_1(\mathbf{x})$. When the scores are non-negative and sum to one they can be read as class probabilities, and Note 1, Section 4, showed that predicting the most probable class minimises the expected classification error. The question of this note is how to obtain such probabilities.

2 The Bayes classifier

Write the posterior with Bayes' rule (Note 2, Section 1.1):

$$P(Y = c \mid \mathbf{X} = \mathbf{x}) = \frac{p(\mathbf{x} \mid Y = c)\, P(Y = c)}{\sum_{c'=1}^{m} p(\mathbf{x} \mid Y = c')\, P(Y = c')}. \tag{1}$$

The *class-conditional density* $p(\mathbf{x} \mid Y = c)$ says what examples of class c look like; the *prior* $P(Y = c)$ says how common the class is. The *Bayes classifier* estimates both from the training set and plugs the estimates into (1):

1. split D_{train} by class into subsets of sizes $n_1, \dots, n_m$;
2. fit a density estimator $\hat{p}_c(\mathbf{x})$ to each subset;
3. estimate priors by class frequencies, $\hat{P}_c = n_c / n$;
4. predict $f(\mathbf{x}) = \arg\max_c \hat{p}_c(\mathbf{x})\, \hat{P}_c$, or report the normalised posteriors.

The denominator of (1) is the same for every class, so it matters only when probabilities, not just decisions, are needed. Any density estimator can be used in step 2 — a Gaussian (Note 2), a histogram (Note 3), a kernel density estimate (Note 5) or a deep generative model — and the classifier inherits

its capacity and its weaknesses. With kernel density estimates the Bayes classifier is close to the Parzen-window classifier of Note 5; with d large, it inherits the curse of dimensionality of Note 3.

Compute in log space. Densities in high dimension underflow (Note 2, Section 6). Work with $s_c(\mathbf{x}) = \log \hat{p}_c(\mathbf{x}) + \log \hat{P}_c$; the decision is $\arg\max_c s_c$, and the posteriors are $\exp(s_c - s_{\max}) / \sum_{c'} \exp(s_{c'} - s_{\max})$, the softmax of the log-scores with the largest subtracted first.

The role of the prior. In two-class form, (1) reads

$$\underbrace{\frac{P(Y=1 \mid \mathbf{x})}{P(Y=0 \mid \mathbf{x})}}_{\text{posterior odds}} = \underbrace{\frac{p(\mathbf{x} \mid Y=1)}{p(\mathbf{x} \mid Y=0)}}_{\text{likelihood ratio } \Lambda(\mathbf{x})} \times \underbrace{\frac{P(Y=1)}{P(Y=0)}}_{\text{prior odds}}.$$

The data enter only through the likelihood ratio; the prior shifts the threshold. Figure 1 shows two unit-variance Gaussian classes two and a half standard deviations apart: with equal priors the boundary sits half-way, at $x = 1.25$; if the signal makes up 1% of the data, it moves to $x \approx 3.1$, and most of the signal is classified as background. This is correct — it minimises the number of errors — but it may not be what we want, which is why decisions with unequal costs move the threshold again (Exercise E2). Because the two factors separate, a classifier trained with one class balance can be corrected for another by rescaling its odds, provided the class-conditional densities are unchanged [7].

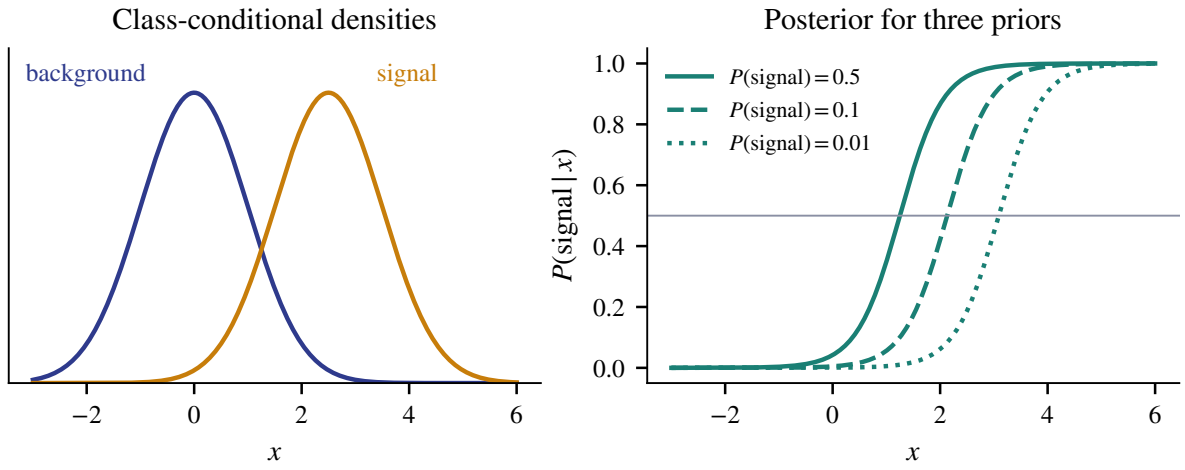


Figure 1. The same class-conditional densities (left) give different posteriors (right) under different priors. The decision boundary is where the posterior crosses $\frac{1}{2}$.

> **In physics — the Neyman–Pearson lemma and learned likelihood ratios**
>
> In a search for a rare process the prior is tiny and largely unknown, so physicists rank events by the likelihood ratio $\Lambda(\mathbf{x})$ itself. The Neyman–Pearson lemma states that, for any fixed rate of falsely accepted background, the test that accepts events with large $\Lambda(\mathbf{x})$ has the highest signal efficiency. The class densities are rarely known in closed form, but a well-calibrated classifier trained on simulated signal and background events estimates $P(Y = 1 \mid \mathbf{x})$, and the identity above turns it back into $\Lambda(\mathbf{x})$. This "likelihood-ratio trick" lets neural networks stand in for intractable likelihoods in parameter inference [2].

3 Gaussian class densities

Let each class be Gaussian, $p(\mathbf{x} \mid Y = c) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_c, \Sigma_c)$, with parameters fitted by the class means and covariances (Note 2, Section 6). The log-score is

$$s_c(\mathbf{x}) = -\tfrac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_c)^{\mathsf{T}}\Sigma_c^{-1}(\mathbf{x} - \boldsymbol{\mu}_c) - \tfrac{1}{2}\log|\Sigma_c| + \log P_c + \text{const},$$

a quadratic function of $\mathbf{x}$, so the boundary $s_1(\mathbf{x}) = s_0(\mathbf{x})$ is a quadric: an ellipse, a parabola or a hyperbola in two dimensions. This is *quadratic discriminant analysis* (QDA).

If all classes share one covariance Σ (estimated by pooling the classes), the quadratic terms $-\tfrac{1}{2}\mathbf{x}^{\mathsf{T}}\Sigma^{-1}\mathbf{x}$ are identical and cancel in every comparison. What remains is linear:

$$s_1(\mathbf{x}) - s_0(\mathbf{x}) = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b, \qquad \mathbf{w} = \Sigma^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0), \qquad b = -\tfrac{1}{2}\big(\boldsymbol{\mu}_1^{\mathsf{T}}\Sigma^{-1}\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0^{\mathsf{T}}\Sigma^{-1}\boldsymbol{\mu}_0\big) + \log\frac{P_1}{P_0}. \tag{2}$$

This is *linear discriminant analysis* (LDA): the decision boundary is a hyperplane. With equal priors the rule assigns $\mathbf{x}$ to the class whose mean is nearest in Mahalanobis distance; if moreover $\Sigma = \sigma^2 I$, to the nearest mean in Euclidean distance — the nearest-centroid classifier of Note 9. Figure 2 shows both cases.

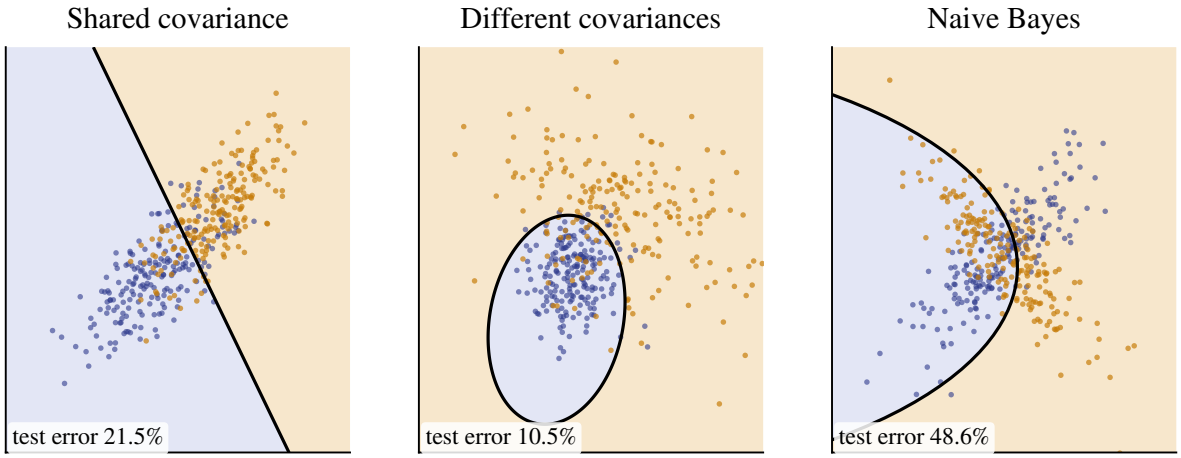


Figure 2. Gaussian Bayes classifiers fitted to 200 points per class (indigo and amber), errors on 10^4 fresh points. Left: shared covariance gives a straight boundary. Centre: a compact class inside a broad one needs the quadratic boundary of QDA. Right: the two classes have the same mean and variances and differ only in the sign of their correlation; naive Bayes cannot see the difference.

Counting parameters. QDA in d dimensions estimates m covariance matrices of $d(d+1)/2$ entries each; LDA one; a diagonal model md variances. For $d = 100$ and $m = 10$ these are about 50 000, 5 000 and 1 000 numbers. More flexible is not automatically better: with few examples per class the shared, shrunk covariance of LDA often generalises best, an instance of the capacity trade-off of Note 4.

4 Naive Bayes

The *naive Bayes* classifier assumes that, within each class, the features are independent:

$$p(\mathbf{x} \mid Y = c) = \prod_{j=1}^{d} p_j(x_j \mid Y = c), \qquad s_c(\mathbf{x}) = \log P_c + \sum_{j=1}^{d} \log \hat{p}_{c,j}(x_j). \tag{3}$$

Instead of one density in d dimensions, it estimates d densities in one dimension — a task with no curse of dimensionality, for which histograms or kernel estimates work well. Training is a single pass over the data, and features of different types (counts, categories, real values) can be mixed freely.

The price is capacity. The model cannot represent any interaction between features within a class: in the right-hand panel of Figure 2, where the classes differ only through the correlation between x_1 and x_2, naive Bayes has 48.6% test error, no better than guessing, while a Gaussian Bayes classifier with full covariances reaches 20.9% on the same data. Two warnings follow. "Bayes classifier" and "naive Bayes" are not synonyms: naive Bayes is the special case with factorised class densities, and its capacity is low, whereas a Bayes classifier with a flexible density estimator can have very high capacity. And naive Bayes is often a better *classifier* than its *probabilities* suggest: correlated evidence is counted several times, so its posteriors are overconfident even when its decisions are good.

Text and counts. For a document represented by word counts n_j (Note 1, Section 3), the multinomial naive Bayes model gives $s_c = \log P_c + \sum_j n_j \log \pi_{c,j}$, where $\pi_{c,j}$ is the frequency of word j in class c. A word never seen in a class would get $\pi_{c,j} = 0$ and veto the class forever — the zero-count problem of Note 2, Exercise E1. Adding a pseudo-count, $\hat{\pi}_{c,j} = (n_{c,j} + \alpha)/(n_c + \alpha V)$ with vocabulary size V, removes it (*Laplace smoothing*).

> **Modern connection — a baseline that refuses to die**
>
> Spam filters of the early 2000s were naive Bayes classifiers on word counts, and the method remains a strong baseline for text: on sentiment and topic benchmarks, naive Bayes features combined with a linear classifier rivalled far more complex models of the time [8]. Today the first thing to try on a small labelled text dataset is a linear classifier on embeddings from a pre-trained model (Note 9); comparing against naive Bayes on raw counts shows how much the representation actually adds.

5 Generative and discriminative classifiers

The Bayes classifier is *generative*: it models how the inputs of each class are generated, i.e. the joint distribution $P(\mathbf{X}, Y) = p(\mathbf{x} \mid Y)\, P(Y)$, and derives $P(Y \mid \mathbf{x})$ from it. Equivalently one may estimate the joint distribution directly and normalise, $P(Y = c \mid \mathbf{x}) = P(\mathbf{x}, c) / \sum_{c'} P(\mathbf{x}, c')$. A *discriminative* classifier, such as logistic regression (Note 9), models $P(Y \mid \mathbf{x})$ directly and never describes the inputs. The trade-off is classical. Generative models make stronger assumptions, need fewer examples to reach their best performance, handle missing features by marginalisation, and can flag inputs with low $p(\mathbf{x})$ as unusual; discriminative models spend all their capacity on the decision and usually win when data are plentiful. For naive Bayes against logistic regression, the generative model approaches its (higher) asymptotic error faster, the discriminative one reaches a lower error eventually [6]. LDA and logistic regression make the point sharply: both produce a linear boundary $\mathbf{w}^{\mathsf{T}}\mathbf{x} + b$, but LDA reads $\mathbf{w}$ off fitted Gaussians via (2), while logistic regression fits $\mathbf{w}$ to the labels.

> **Modern connection — generative models as classifiers**
>
> Every model that assigns a likelihood to data can be turned into a classifier by (1). A text-to-image diffusion model (Note 2, Section 5) estimates how well an image fits each candidate caption; choosing the caption with the best fit gives a zero-shot image classifier with surprisingly strong results, including robustness to some distribution shifts [4]. Language models are used the same way when the probability of each label text, given the input, is compared. These

are Bayes classifiers with billions of parameters in the density estimator.

Summary The Bayes classifier fits a density to each class and a prior, and combines them with Bayes' rule; posterior odds equal the likelihood ratio times the prior odds. Gaussian class densities give quadratic boundaries (QDA), or linear ones when the covariance is shared (LDA), which reduces to nearest centroid in the simplest case. Naive Bayes assumes independent features within each class: one-dimensional estimates, no curse, low capacity and overconfident probabilities. Generative classifiers model the inputs; discriminative ones model only the decision.

Exercises

E1. Derive (2) from the Gaussian log-scores with a shared covariance. Show that for $\Sigma = \sigma^2 I$ and equal priors the boundary is the perpendicular bisector of the segment joining $\boldsymbol{\mu}_0$ and $\boldsymbol{\mu}_1$.

E2. Missing a signal event costs C_1 and accepting a background event costs C_0. Show that the decision minimising the expected cost is to predict signal when $\Lambda(\mathbf{x}) > (C_0 P_0)/(C_1 P_1)$. Where is the boundary of Figure 1 for $P_1 = 0.01$ and $C_1 = 100\, C_0$?

E3. For two one-dimensional Gaussian classes with different variances, show that the decision region of the class with the smaller variance is an interval. When is it empty?

E4. Implement a Gaussian naive Bayes and a full-covariance Gaussian Bayes classifier and reproduce the right-hand panel of Figure 2. Then rotate the data by 45°: what happens to naive Bayes, and why?

E5. A spam filter uses multinomial naive Bayes with $V = 20\,000$ words. How many parameters does it estimate per class? A message contains a word seen only in legitimate mail. Compute the effect on the log-odds with and without Laplace smoothing ($\alpha = 1$).

E6. A classifier was trained on simulated events with equal numbers of signal and background but will be applied to data in which signal is 0.1% of events. Write the corrected posterior in terms of the classifier output $g(\mathbf{x})$.

Further reading. Bishop [1], sections 4.1.4 and 4.2 (Fisher's discriminant and probabilistic generative models); Hastie, Tibshirani and Friedman [3], sections 4.3 and 6.6.3; Murphy [5], chapter 9.

References

[1] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer (2006).

[2] K. Cranmer, J. Pavez and G. Louppe, Approximating likelihood ratios with calibrated discriminative classifiers, arXiv:1506.02169 (2015).

[3] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer (2009).

[4] A. C. Li, M. Prabhudesai, S. Duggal, E. Brown and D. Pathak, Your diffusion model is secretly a zero-shot classifier, *ICCV* (2023).

[5] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press (2022).

[6] A. Y. Ng and M. I. Jordan, On discriminative vs. generative classifiers: a comparison of logistic regression and naive Bayes, *NeurIPS* (2002).

[7] M. Saerens, P. Latinne and C. Decaestecker, Adjusting the outputs of a classifier to new a priori probabilities: a simple procedure, *Neural Comput.* 14, 21–41 (2002).

[8] S. Wang and C. D. Manning, Baselines and bigrams: simple, good sentiment and topic classification, *ACL* (2012).

The selection and order of topics in these notes follow the course IFT 6390 *Fondements de l'apprentissage machine* at Université de Montréal (taught by Ioannis Mitliagkas, slides by Pascal Vincent). The text, examples, figures and exercises of this note are original. Figures are computed from synthetic data by `figures.py`, published with the sources of these notes.